



Buffer Overflow Attack

Buffer Overflow



- **Buffer:** contiguous allocated chunk of memory, such as an array or a pointer in C.
- Buffer overflow occurs when a user or a program writes past the buffer, intruding into memory locations not allocated for the buffer.
- Programming languages like C and C++ provide no automatic bound checking on buffers thereby allowing user to write past the buffer.

2

Anatomy of a Buffer Overflow

- *Buffer*: memory used to store user input, has fixed maximum size
- *Buffer overflow*: when user input exceeds max buffer size
 - Extra input goes into unexpected memory locations

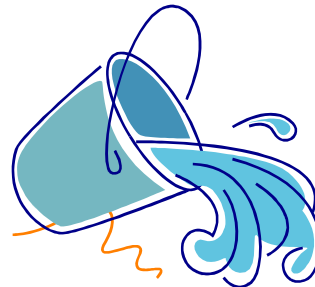


3

A Small Example

- Malicious user enters > 1024 chars, but `buf` can only store 1024 chars; extra chars overflow buffer

```
void get_input() {  
    char buf[1024];  
    gets(buf);  
}  
void main(int argc, char*argv[]){  
    get_input();  
}
```



4

The Problem

```
void try1(char *s) {  
    char buf[10];  
    strcpy(buf,s);  
    printf("buf is %s\n",s);  
}  
...  
try1("thisstringistolong");
```

5

Continued.



```
int main()  
{  
    int buffer[10];  
    buffer[20]=10;  
}
```

Writing past the buffer

- Any C compiler will compile the above segment with no errors.

6

Buffer Overflows



- Applications running on the host in Internet have certain privileges.
- They can access part of the system resources like system variables, system files and even execute certain commands that a remote user/application cannot.
- From an attacker's view point if he can take control of a vulnerable application running on target on a target computer he can misuse the privileges of victim application.
- This happens in buffer overflow.

7

Types of Buffer Overflows

- Stack Overflows
- Format String Overflows
- Heap Overflows
- Integer Overflows



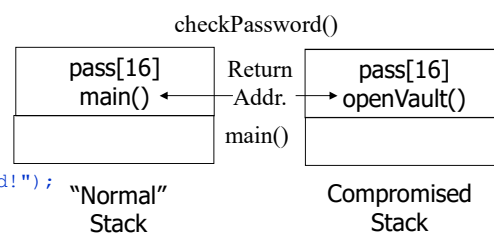
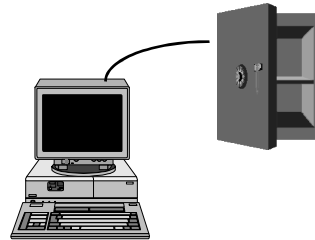
8

A More Detailed Example

```

1 int checkPassword() {
2     char pass[16];
3     bzero(pass, 16); // Initialize
4     printf ("Enter password: ");
5     gets(pass);
6     if (strcmp(pass, "opensesame") == 0)
7         return 1;
8     else
9         return 0;
10 }
11
12 void openVault() {
13     // Opens the vault
14 }
15
16 main() {
17     if (checkPassword()) {
18         openVault();
19         printf ("Vault opened!");
20     }
21 }

```



checkPassword() Bugs

- *Execution stack*: maintains current function state and address of return function
- *Stack frame*: holds vars and data for function
- Extra user input (> 16 chars) overwrites return address
 - *Attack string*: 17-20th chars can specify address of `openVault()` to bypass check
 - Address can be found with source code or binary

The safe_gets() Function

```
#define EOLN '\n'
void safe_gets (char *input, int max_chars) {
    if ((input == NULL) || (max_chars < 1)) return;
    if (max_chars == 1) { input[0] = 0; return; }
    int count = 0;
    char next_char;
    do {
        next_char = getchar(); // one character at a time
        if (next_char != EOLN)
            input[count++] = next_char;
    } while ((count < max_chars-1) && // leave space for null
              (next_char != EOLN));
    input[count]=0;
}
```

- Unlike gets () , takes parameter specifying max chars to insert in buffer
- Use in checkPassword () instead of gets () to eliminate buffer overflow vulnerability

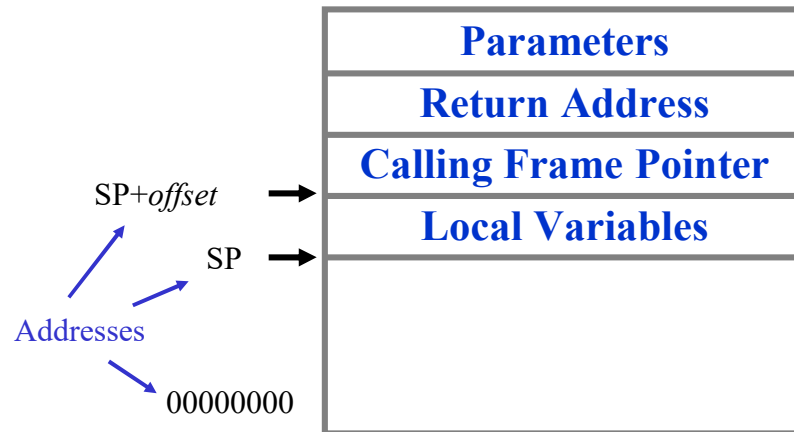
11

Stack Direction

- On Linux (x86) the stack grows from high addresses to low.
- Pushing something on the stack moves the Top Of Stack towards the address 0.

12

A Stack Frame



13

Sample Stack

18
<i>addressof(y=3) return address</i>
<i>saved stack pointer</i>
<i>y</i>
<i>x</i>
<i>buf</i>

```

x=2;
test1(18);
y=3;

void test1(int j) {
    int x,y;
    char buf[100];
    x=j;
    ...
}

```

Blue arrows indicate the flow of control and data:

- An arrow points from the `test1(18);` call to the `void test1(int j) {` function definition.
- Another arrow points from the `y=3;` statement to the `int x,y;` declaration inside the function.

14

Return-Oriented Program (ROP)

- A Return-Oriented Program (ROP) is an attack technique used in computer security to execute malicious behavior without injecting new code.
- Return-Oriented Programming (ROP) is a code-reuse attack where an attacker:
 - Reuses existing instructions already present in a program or its loaded libraries.
- ROP Chain
 - A ROP chain is a sequence of gadget addresses placed on the stack.
 - Each RET jumps to the next gadget.

15

EBP (Extended Base Pointer)

- 💡 **EBP is a register used to mark the base of the current stack frame.**
- **It helps access local variables and function parameters.**
- **Think of EBP as a fixed reference point inside the stack.**
- **While the Stack Pointer (ESP) keeps changing (push/pop), the CPU uses EBP as a stable anchor.**
- 💡 **Analogy:**
 - ESP: An elevator (moves up and down).**
 - EBP: A floor marker (fixed).**

Accessing Variables via EBP

Stack Layout After Setup:

Stack Layout (High to Low Memory)

Function Args (Accessed via positive offset).

Return Address: Located at [EBP + 4]

Old EBP: Located at [EBP]

Low Memory: Local Variables (Accessed via negative offset).

Access Patterns:

Arguments:

- [EBP + 8] (1st Arg),
- [EBP + 12] (2nd Arg)

Locals:

- [EBP - 4] (1st Var)
- [EBP - 8] (2nd Var)

C Code & Stack View

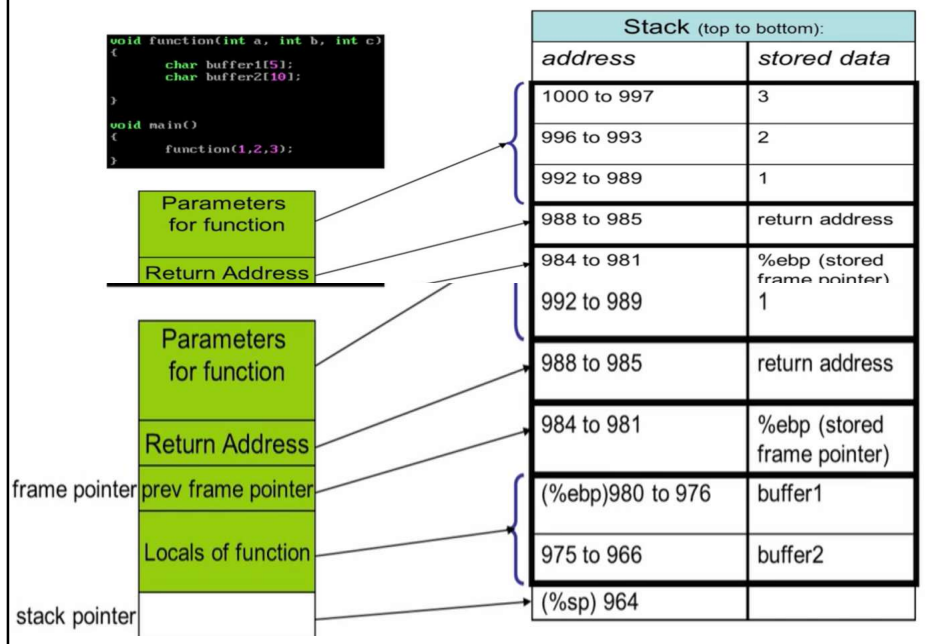
C Code :

```
void add(int a, int b) {
    int sum = a + b;
}
```

Mental Stack Model:

- ◆ EBP + 12 → b (Argument 2)
- ◆ EBP + 8 → a (Argument 1)
- ◆ EBP + 4 → Return Address
- ◆ EBP → Old EBP
- ◆ EBP - 4 → sum (Local Variable)

Stack Usage (example)



Stack Usage Contd.

```
void function(int a, int b, int c)
{
    char buffer1[5];
    char buffer2[10];
}

void main()
{
    function(1,2,3);
}
```

What is the output of the following?

- `printf("%x", buffer2) : 966`
- `printf("%x", &buffer2[10])`

What is the output of the following?

- `printf("%x", buffer2) : 966`
- `printf("%x", &buffer2[10])`
976 → buffer1

Therefore `buffer2[10] = buffer1[0]`

A BUFFER OVERFLOW

Stack (top to bottom):	
address	stored data
1000 to 997	3
996 to 993	2
992 to 989	1
988 to 985	return address
984 to 981	%ebp (stored frame pointer)
992 to 989	1
988 to 985	return address
984 to 981	%ebp (stored frame pointer)
(%ebp)980 to 976	buffer1
975 to 966	buffer2
(%sp) 964	

Modifying the Return Address

buffer2[19] =
&arbitrary memory location

Stack (top to bottom):	
address	stored data
1000 to 997	3
996 to 993	2
992 to 989	1
988 to 985	Arbitrary Location
984 to 981	%ebp (stored frame pointer)
(%ebp)980 to 976	buffer1
976 to 966	buffer2
(%sp) 964	

19

gadget addresses

- A gadget is a small sequence of existing instructions already present in a program (or linked libraries) that ends with a ret instruction.
- A gadget address is simply the memory address where that gadget starts.
- 0x080484a0 → pop eax ; ret
- So:
 - Gadget = pop eax ; ret
 - Gadget address = 0x080484a0

22

gadget addresses matter in buffer overflow

- In a buffer overflow, the attacker can overwrite the return address on the stack.
- Instead of returning to normal code, the program:
 - Returns to a gadget address
 - Executes that gadget
 - The gadget ends with ret
 - Control jumps to the next gadget address

This forms a ROP chain.

23

Normal program execution (NO attack)

- Stack before function returns
- **Program returns normally to main()**

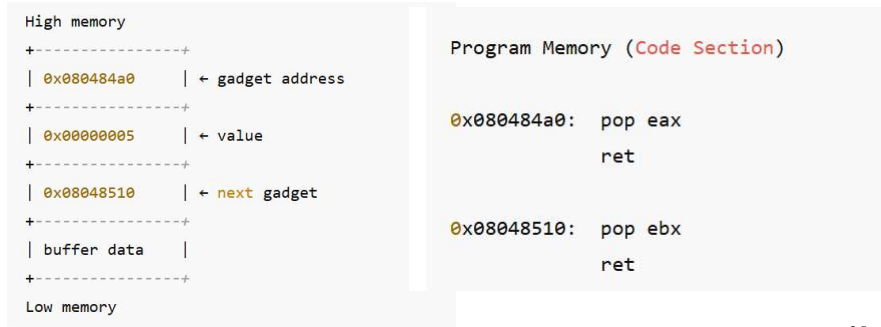
```

High memory
+-----+
| Return Addr | → main()
+-----+
| Saved EBP   |
+-----+
| buffer[32]  |
+-----+
Low memory
  
```

24

Buffer overflow happens

- Unsafe input overwrites return address.
- Return address replaced with gadget address.



25

Execution flows (step-by-step)

1. RET → 0x080484a0

– pop eax

– ret

2. RET jumps again)

RET → 0x08048510

Executes

– pop ebx

– Ret

Stack:

[0x080484a0] → pop eax ; ret

[0x00000005]

[0x08048510] → pop ebx ; ret

[0x00000000]

Execution:

RET → Gadget 1 → RET →

Gadget 2 → RET → ...

A gadget address is the memory location of a small instruction sequence ending with ret, used in buffer overflow attacks to control program execution step by step.

26

Introducing ChatGPT | OpenAI | Buffer Overflow Examples | Online C Compiler | sigsegr - Google Search

codechef.com/c-online-compiler

```

1 #include <stdio.h>
2 #include <string.h>
3
4 void login() {
5     char password[10];
6     int isAuthenticated = 0; // 0 = Not authenticated, 1 = Authenticated
7
8     printf("Enter password: ");
9     gets(password); // Unsafe! Buffer overflow vulnerability
10
11     if (strcmp(password, "secret123") == 0) {
12         isAuthenticated = 1; // Correct password sets authentication flag
13     }
14
15     if (isAuthenticated) {
16         printf("Access granted! Welcome to the system.\n");
17     } else {
18         printf("Access denied! Incorrect password.\n");
19     }
20 }
21
22 int main() {
23     login();
24     return 0;
25 }
26

```

deepak singh tomar aaja hack kare kyo aaj kal

Output

Status: Runtime error

Time: 0.0000 secs Memory: 1.74 Mb

Sample Input

deepak singh tomar aaja hack kare kyo aaj kal

Runtime Error

SIGSEGV

Type here to search 16°C Haze 06:16 10-02-2025

OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.

IDE

My Projects

Classroom **new**

Learn Programming

Programming Questions

Sign Up

Login

About • FAQ • Blog • Terms of Use • Contact
Us • GDB Tutorial • Credits • Privacy

main.c

```

6 int isAuthenticated = 0; // 0 = Not authenticated, 1 = Authenticated
7
8 printf("Enter password: ");
9 gets(password); // Unsafe! Buffer overflow vulnerability
10
11 if (strcmp(password, "secret123") == 0) {
12     isAuthenticated = 1; // Correct password sets authentication flag
13 }
14
15 if (isAuthenticated) {
16     printf("Access granted! Welcome to the system.\n");
17 } else {
18     printf("Access denied! Incorrect password.\n");
19 }
20 }
21
22 int main() {
23     login();
24     return 0;
25 }
26

```

input

Access denied! Incorrect password.
*** stack smashing detected ***: terminated

...Program finished with exit code 134
Press ENTER to exit console.

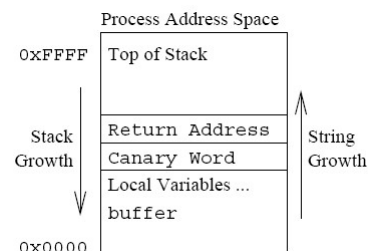
Safe String Libraries

- C – Avoid (no bounds checks): **strcpy()**, **strcat()**, **sprintf()**, **scanf()**
- Use safer versions (with bounds checking): **strncpy()**, **strncat()**, **fgets()**
- Must pass the right buffer size to functions!
- C++: STL string class handles allocation
- Unlike compiled languages (C/C++), interpreted ones (Java/C#) enforce type safety, raise exceptions for buffer overflow

29

StackGuard

- *Canary*: random value, unpredictable to attacker
- Compiler technique: inserts *canary* before return address on stack
- Corrupt Canary: code halts program to thwart a possible attack
- Not comprehensive protection



Source: C. Cowan et. al., StackGuard,

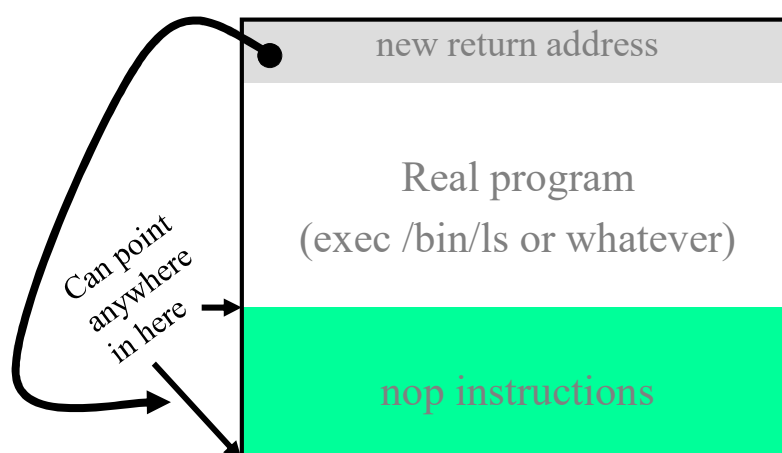
30

NOPs

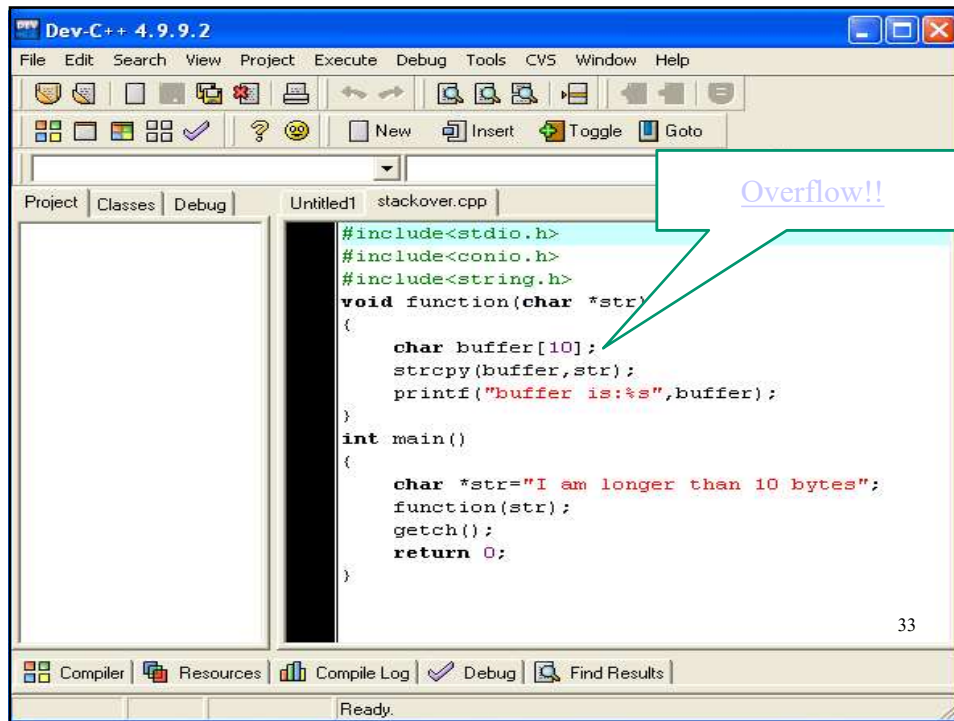
- Most CPUs have a *No-Operation* instruction – it does nothing but advance the instruction pointer.
- Usually we can put a bunch of these ahead of our program (in the string).
- As long as the new return-address points to a NOP we are OK.

31

Using NOPs



32



Dev-C++ 4.9.9.2

File Edit Search View Project Execute Debug Tools CVS Window Help

Project Classes Debug Untitled1 stackover.cpp

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void function(char *str)
{
    char buffer[10];
    strcpy(buffer, str);
    printf("buffer is:%s", buffer);
}
int main()
{
    char *str="I am longer than 10 bytes";
    function(str);
    getch();
    return 0;
}
```

Overflow!!

33

Compiler Resources Compile Log Debug Find Results

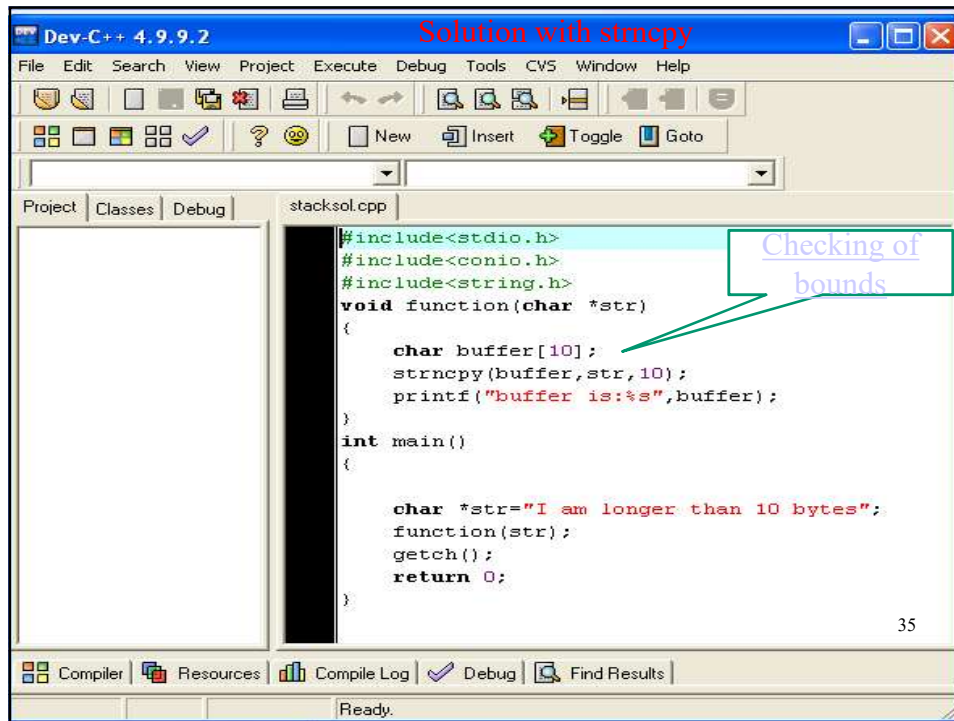
Ready.

Solutions



- The reason behind stack smashing is mismanagement of memory and lazy programming.
- The main reason behind stack smashing is that no bound checking is performed.
- Instead of using functions like strcpy, strcat use strncpy and strncat which performs bound checking.

34



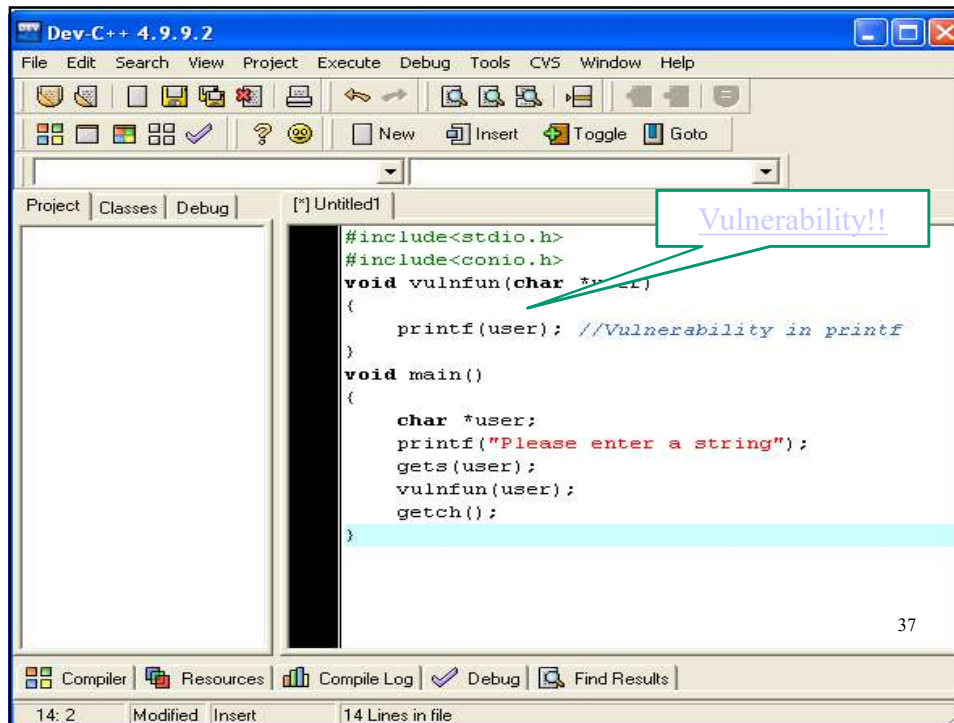
Format string overflows

- This type of vulnerability occurs while using the format functions in C.
- Examples for format functions are:
 - printf- printing to stdout
 - fprintf- printing to a file
 - sprintf- printing to a string pointer

Format functions typically have 2 arguments:

- Format string like %d, %s, %x representing the type and no: of parameters that are to follow.
- Data that is to be sent to the output stream
- Eg: printf(“%d”,input_decimal)

36



The screenshot shows the Dev-C++ 4.9.9.2 IDE. The main window displays a C program named 'Untitled1'. The code includes `<stdio.h>` and `<conio.h>`. It defines a function `vulnfun(char *user)` which calls `printf(user);` with a comment `//Vulnerability in printf`. The `main` function prompts the user to enter a string, reads it with `gets(user)`, and then calls `vulnfun(user)`. A red box with the text 'Vulnerability!!' points to the `printf` statement in the `vulnfun` function. The status bar at the bottom indicates '14: 2 Modified Insert 14 Lines in file'.

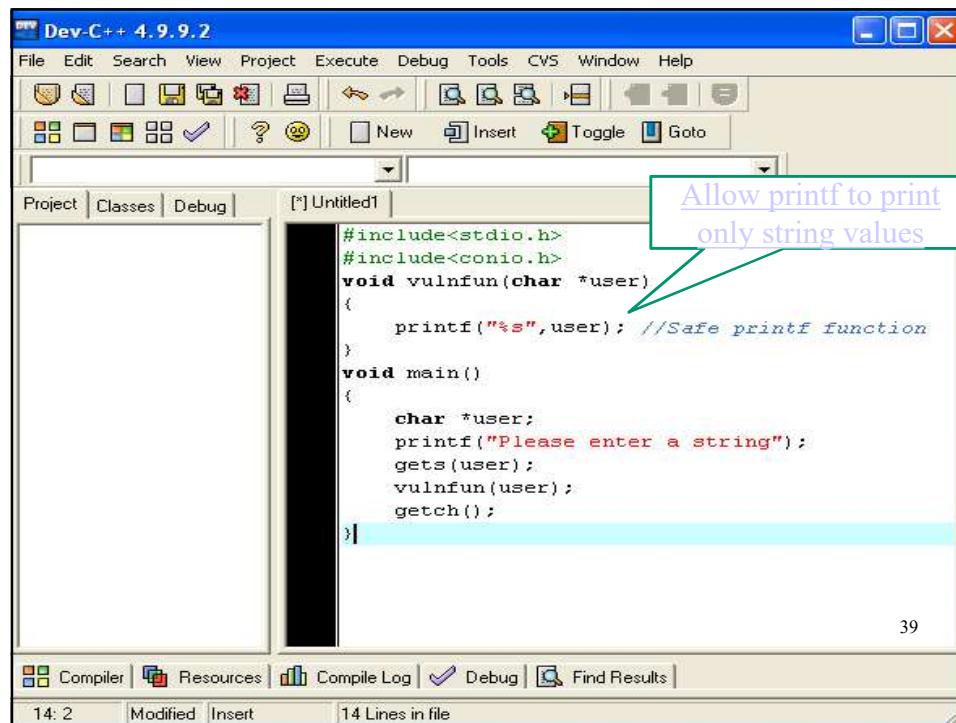
```
#include<stdio.h>
#include<conio.h>
void vulnfun(char *user)
{
    printf(user); //Vulnerability in printf
}
void main()
{
    char *user;
    printf("Please enter a string");
    gets(user);
    vulnfun(user);
    getch();
}
```

Continued...



- Instead of giving a string if we give a format string as below for the input of the `vulnfun()` function:
`"%08x.%08x.%08x.%08x.%08x"`
- The `printf` below prints the values of five consecutive words in the stack (as eight-digit padded hexadecimal numbers). We can walk up the stack this way and view the contents of the stack.

38



Stack Walking with Format Strings

The Tool: %08x

- 💡(%x): Print a value from the stack in hexadecimal.
- 💡(8): Total width of 8 characters.
- 💡(0): Pad with zeros.
- 💡Result: Prints one 32-bit stack value (e.g. : 0000002f).

The Chain:

- 💡Using `%08x.%08x.%08x` reads sequential 4-byte values from the stack.
- 💡This "walks" through the stack step by step.

Vulnerability Mechanism

Unsafe Code: `printf(user_input);`

The Flaw:

- If the user inputs format specifiers (e.g., `%08x`), `printf` expects arguments .
- Since no arguments are passed, it reads whatever is currently on the stack.

The Walk:

- 1st `%08x`: Reads Return Address.
- 2nd `%08x`: Reads Saved EBP.
- Subsequent `%08x`: Reads internal stack data.

Output & Impact

Typical Output:

- `080484f0.bffff2c8.00000001.b7fd0ff4...`
- Values are printed in hex, zero-padded, revealing stack locations .

Why "Stack Walking"?

- It moves forward on the stack without crashing the program.
- It causes Information Leakage, revealing the stack layout, return addresses, and EBP location.

Summary & Mitigation

💡 **Definition:** `%08x` in a format string reads and prints successive 32-bit values from the stack, allowing attackers to inspect memory.

💡 **Key Learning Point:**

Dangerous: `printf(user_input)`

Safe: `printf("%s", user_input)`

Countermeasures

- Secure coding:
 - Bound checking mandatory
 - File access and user permissions must be kept in mind.
 - Access permissions of even temporary files should be restricted to prevent any malicious modifications in between two consecutive executions
 - All user inputs should be checked or validated for malicious code.



44

Stop execution of malicious commands



- Designate the data segment of an applications buffer space as non executable zone.
- Unfortunately latest versions of unix & windows do not have such a facility .
- Depending on the platform it is possible to download a patch from your vendor.
- But this has the disadvantage that it stops legitimate applications from offering there full fledged feauters.⁴⁵ It is a matter of balance between security & functionality.

Conclusion



- Buffer Overflow attacks are not only very dangerous but once executed,they are also quite easy to execute.
- It is very important to take adequate security countermeasures against buffer overflows.
- It is suggested to use programming languages like java which do not give direct access on addresses.
- Buffer Overflow attacks can be countered by Programming Applications efficiently & securely. ⁴⁶

References

- Network Security: A Hacker's Perspective-Ankit Fadia
- https://www.owasp.org/index.php/Buffer_Overflow
- www.linuxjournal.com/article/6701
- www.windowsecurity.com/.../analysis_of_buffer_overflow_attacks
- Operating System #37 Buffer Overflow Attacks Explained in Detail Best Programming Courses @ <https://goo.gl/MVVDXR> Complete Operating Systems Lecture/ Tutorials from IIT @ <https://goo.gl/GMr3if>

47